

06 — Misurare un mazzo: normalizzare, calibrare, non mentire

Come si costruisce un punteggio che significhi qualcosa, quando i dati sono incompleti e la scala se la sceglie chi scrive il codice. Esempi: `src/engine/forza.js`, `src/engine/bilancia.js`, `tests/forza.test.js`.

1. Il problema: due domande diverse, due misure diverse

Il motore aveva già un punteggio, `punteggioMazzo()` in `bilancia.js`. Serve a rispondere a: **fra questi due mazzi che ho appena generato, ce n'è uno più forte?** Somma PS totali e gradini evolutivi totali, e va benissimo, perché i mazzi che confronta hanno la stessa taglia per costruzione.

La domanda nuova è un'altra: **il mazzo che ho appena generato regge il Kit Allenatore che sta nella scatola in salotto?** Quel Kit è da 30 carte, o da 60 se si uniscono i due mazzetti, e il mazzo generato può essere da 15.

Con una misura che **somma**, un mazzo da 60 batte sempre uno da 30:

```
punteggioMazzo(trenta).totale; // 118
punteggioMazzo(sessanta).totale; // 236 ← stesso mazzo, raddoppiato
```

Non è un bug: è che quella misura risponde a un'altra domanda. Ecco perché il progetto ora ha **due moduli** invece di uno modificato — e il test che lo dice a voce alta:

```
test('è proprio ciò che punteggioMazzo non sa fare (controllo)', async () => {
  const { punteggioMazzo } = await import('../src/engine/bilancia.js');
  assert.ok(punteggioMazzo(sessanta()).totale > punteggioMazzo(trenta()).totale * 1.5);
});
```

Nota di metodo. La tentazione era riscrivere `punteggioMazzo()`. Ma quella funzione è il criterio su cui è tarato l'hill-climbing di `bilancia()`, con una soglia (`SOGLIA_SQUILIBRIO`) trovata provando, e dieci test addosso. Cambiarla per una funzione nuova avrebbe rimesso in discussione un pezzo che funziona. **Aggiungere accanto costa meno che modificare sotto.**

2. Normalizzare: media, non somma

La regola che rende una misura indipendente dalla taglia è banale a dirsi: ogni indicatore è una **media per carta**, mai un totale.

```
const psMedi = pokemon.reduce((s, c) => s + (c.carta.ps ?? 0) * c.quantita, 0) / copie;
const resistenza = limita(psMedi / TETTI.ps);
```

Il test che protegge la proprietà è il più importante del file, ed è scritto come proprietà, non come valore atteso:

```
test('la forza non dipende dalla taglia: 30 e 60 con le stesse proporzioni si equivalgono', () => {
  assert.ok(Math.abs(forza(trenta()).totale - forza(sessanta()).totale) <= 2);
});
```

Non asserisce *quanto* vale un mazzo — quel numero cambierà ogni volta che si ritocca un peso, e il test diventerebbe un freno. Asserisce **una relazione fra due risultati**, che deve valere sempre.

È la differenza fra un test di regressione e un test di proprietà. Chi arriva da JUnit la conosce come il salto da `assertEquals(118, punteggio)` a `assertThat(a).isCloseTo(b)`: il primo si rompe a ogni taratura, il secondo si rompe solo quando si rompe l'idea.

3. Calibrare: da dove viene il numero che vale 100

Normalizzare vuol dire dividere per qualcosa. Quel qualcosa è una decisione, ed è il punto in cui un punteggio diventa arbitrario senza che si veda.

L'istinto è normalizzare sul massimo: il Pokémon più resistente del dataset ha 380 PS, quindi $380 = 100$. Il risultato sarebbe che ogni mazzo di casa vale 25 e la scala non distingue più niente, perché i valori estremi appartengono a poche carte da torneo che in una collezione di famiglia non esistono.

La scelta è stata **misurare il dataset** e prendere il 90° percentile:

indicatore	mediana	p90 (= tetto scelto)	massimo
danno per Energia	25	55	250
PS	100	220	380

Il codice che li ha prodotti è tre righe di `node` — e sui 12.877 Pokémon in `data/set/`. Non è finito nel repo perché è servito una volta sola — ma il numero che ne è uscito è **nel commento**, con la sua ragione:

```
export const TETTI = { dannoPerEnergia: 55, ps: 220 };
```

La regola generale: quando un valore costante decide il comportamento, nel codice deve finire anche *come lo hai trovato*. Fra sei mesi 55 senza commento è indistinguibile da un numero inventato, e nessuno oserà toccarlo.

4. Dati mancanti ≠ valore zero

Il dataset è completo al 98,9%. Ma i set Kit Allenatore — **proprio quelli che servono da metro** — sono ristampe, e TCGdex non vi replica i dati di gioco:

copertura degli attacchi	dataset intero	tk-sm-r	tk-sm-l	tk-xy-b
Pokémon con almeno un attacco	98,9 %	33 %	17 %	0 %

Ci sono tre modi di reagire, e due sono sbagliati.

Sbagliato 1 — contare zero. Un Pokémon senza dati non è un Pokémon che non attacca. Contarlo zero nella media dice «mazzo debole» quando la verità è «non lo sappiamo», e le due cose portano a decisioni opposte.

Sbagliato 2 — mettere un valore di comodo. Negli attacchi dei Kit il campo `costo` è un array vuoto. `bilancia.js` fa così:

```
(Number(a.danno) || 0) / Math.max(1, a.costo?.length ?? 1)
```

Quel `Math.max(1, ...)` evita la divisione per zero — ma trasforma «costo sconosciuto» in «costa 1», che è il costo *migliore possibile*, e **gonfia** la resa di un fattore due o tre esattamente sulle carte di cui non sappiamo nulla.

Giusto — escludere dalla media e dichiararlo. In `forza.js` la carta senza dati non entra né al numeratore né al denominatore, e il risultato porta con sé quanto ci si può fidare:

```
const copertura = copieMisurate / copie;  
return { ..., attendibile: copertura >= COPERTURA_MINIMA, copertura };
```

La UI legge `attendibile` e scrive «di alcune carte il dataset non ha i dati degli attacchi» invece di mostrare un numero come se fosse una misura.

Chi viene da Java riconoscerà la distinzione fra 0 e null, o fra int e OptionalInt. Qui non c'è un tipo a ricordarla: la disciplina è tutta nel codice, e il posto dove si scrive è il test.

5. Le penalità hanno un verso

L'indicatore `motore` misura se le Energie del mazzo bastano. Prima versione, simmetrica:

```
1 - Math.abs(quota - quotaIdeale) / quotaIdeale
```

Sembrava ragionevole e ha prodotto un bug visibile solo provando l'app vera: un mazzo Lotta con **otto Energia Lotta su trenta carte** — cioè un mazzo giusto — usciva con `motore`: 0. Due errori sommati:

1. il fabbisogno era calcolato sul costo dell'attacco *più redditizio*, che è quasi sempre il più economico, quindi il mazzo sembrava aver bisogno di pochissime Energie;
2. la penalità simmetrica azzerava già al doppio dell'ideale.

La correzione riconosce che **le due direzioni non hanno le stesse conseguenze**: senza Energie non si attacca affatto, con troppe si pesca la carta sbagliata. Un problema è una paralisi, l'altro un fastidio.

```
const adeguatezza = rapporto <= 1 ? limita(rapporto) : limita(1 - (rapporto - 1) / 2);
```

La lezione: prima di scrivere `Math.abs()` in una penalità, chiedersi se sbagliare in eccesso e sbagliare in difetto costino davvero uguale. Quasi mai è così.

6. Un po' di probabilità: l'ipergeometrica

L'indicatore `costanza` risponde a: *qual è la probabilità di avere almeno un Pokémon Base nella mano iniziale?* Senza, non si comincia: si rimescola e si perde il turno.

È la distribuzione **ipergeometrica** — estrazioni *senza* reimmissione, a differenza della binomiale. Si calcola per complemento, perché «almeno uno» è scomodo e «nessuno» è facile:

$$P(\text{almeno una}) = 1 - \frac{\binom{N-K}{n}}{\binom{N}{n}}$$

L'implementazione però **non usa i coefficienti binomiali**:

```
let nessuna = 1;
for (let i = 0; i < mano; i++) nessuna *= (totale - favorevoli - i) / (totale - i);
return 1 - nessuna;
```

Il motivo è pratico: $C(60, 7)$ vale già oltre 386 milioni, e su mazzi più grandi i fattoriali escono dai 2^{53} interi esatti di JavaScript — che, a differenza di Java, non ha `long` né `BigInteger` a portata di mano (c'è `BigInt`, ma è un altro tipo e contagia tutta l'espressione). Moltiplicare frazioni una alla volta tiene i numeri piccoli e il risultato in `double`.

Il test si ancora a un caso verificabile a mente:

```
assert.ok(Math.abs(probabilitaAlmenoUna(30, 1, 7) - 7 / 30) < 1e-12);
```

Con **una** copia sola, «almeno una nelle prime 7 su 30» è esattamente $7/30$. Se la formula fosse sbagliata, questo caso lo direbbe subito — un test che si può rifare a mano vale più di dieci con numeri copiati dall'output.

7. CSS: perché una griglia e non quattro

Le barre della forza sono una in cui ogni è una griglia:

```
.elenco-forza li {  
  display: grid;  
  grid-template-columns: auto minmax(4rem, 1fr) auto;  
}
```

Con quattro elementi in fila (flex) le barre partirebbero da un punto diverso per ogni riga, perché i nomi hanno lunghezze diverse — e confrontarle a occhio, che è **tutto lo scopo del grafico**, non funzionerebbe. La griglia allinea le colonne fra righe diverse: è la cosa che flexbox, per definizione, non sa fare.

Il dettaglio va a capo occupando la riga intera:

```
.forza-dettaglio { grid-column: 1 / -1; }
```

1 / -1 significa «dalla prima all'ultima linea», qualunque sia il numero di colonne: non va aggiornato se un giorno se ne aggiunge una.

Chi arriva da Angular Material troverà familiare l'idea di un layout a colonne dichiarate una volta sola; la differenza è che qui non c'è nessun componente in mezzo — grid-template-columns è l'API.

8. Un metro va costruito, non trovato

Una scala 0–100 non serve a niente finché non c'è qualcosa di noto da confrontarci. «Forza 74» non dice se la partita sarà bella; «74 contro il 31 del Kit di Alola» sì.

Il metro sono i mazzi prefatti, e per costruirlo sono serviti tre pezzi che il dataset **non ha**.

Le quantità non stanno nel catalogo

Un set TCGdex è un catalogo di carte, non un mazzo: dice che il Kit Lycanroc esiste, non che contiene 13 Energia Lotta. Peggio, TCGdex registra 18 delle 30 carte — mancano proprio le Energie.

La lista si scrive a mano in tools/prefatti/, **con la fonte dentro il file**:

```
{ "id": "tk-sm-1", "taglia": 30,  
  "fonte": "https://bulbapedia.bulbagarden.net/wiki/Sun_&_Moon_Trainer_Kit:...",  
  "carte": [{ "numero": "1", "quantita": 1 }, ..., { "energia": "Lotta", "quantita": 13 } ] }
```

Un dato scritto a mano senza la sua provenienza è indistinguibile da un dato inventato — e qui l'invenzione non si vedrebbe, perché produrrebbe comunque un numero plausibile.

I dati di gioco stanno altrove, e vanno riconciliati

Le carte dei Kit sono ristampe, e TCGdex non vi replica attacchi e PS. Ma quelle carte esistono complete nei set normali della stessa epoca, e si ritrovano per nome. Prima versione, ovvia:

```
const gemella = carteDi(idSet).find((c) => normalizza(c.nome) === normalizza(carta.nome));
```

Sbagliata, e in un modo istruttivo. Il Lycanroc del Kit ha **110 PS**; la prima omonima trovata era il promo smp/SM105, che ne ha **120** — stesso nome, stampa diversa, e quindi anche attacchi diversi. Il Kit risultava più forte di quanto è stampato sulle sue carte.

La correzione usa l'unico dato che il Kit dichiara sempre — i PS — come **chiave di riconciliazione**:

```
const stessiPs = omonime.find((o) => carta.ps && o.carta.ps === carta.ps);  
const scelta = stessiPs ?? omonime[0];
```

E quando neanche i PS coincidono, il fatto finisce **nel dato** (`attacchiApprossimati: true`) e nell'output dello strumento, invece di restare un'ipotesi silenziosa. Con questa regola tutte e 24 le carte dei due Kit hanno trovato la stampa giusta, e il Lycanroc viene da `sm3/75`, 110 PS.

È il problema del *record linkage*, familiare a chi ha unito due tabelle su un nome invece che su una chiave. La regola è la stessa ovunque: quando la chiave non è univoca, si cerca un secondo attributo che discrimini, e si dichiara quando non si è trovato.

Un dato generato va testato come codice

`data/mazzi-prefatti.json` lo scrive uno strumento, quindi nessuno lo rilegge. `tests/mazzi-prefatti.test.js` controlla il file **committato**, non lo strumento: le quantità fanno la taglia, ogni Pokémon ha PS e attacchi, le Energie sono riconosciute da `eEnergiaBase()`, e la forza cade nell'intervallo atteso per un mazzo didattico (15–50).

Quest'ultimo è il più utile ed è il meno ovvio: non verifica il catalogo, verifica che **la taratura di `forza()` non sia saltata**. Se un giorno un ritocco ai pesi facesse salire il Kit di Alola a 60, il test lo direbbe subito — e la diagnosi giusta non sarebbe “il Kit è diventato forte”.

9. Centrare un bersaglio senza scrivere un ottimizzatore

Misurare serve a decidere. La decisione qui è: *genera mazzi che valgano quanto il Kit di Alola*, cioè 31.

L'istinto è scrivere un ottimizzatore: parti da un mazzo, scambia carte, ricalcola, ripeti finché il punteggio converge. È codice delicato — si blocca in minimi locali, oscilla, e ogni scambio può rompere una linea evolutiva.

Non è servito, per un motivo che si vede solo **misurando prima di progettare**: `pianifica()` è già seminata, e semi diversi davano forze da 49 a 77 sulla stessa collezione. La dispersione naturale era molto più larga della precisione richiesta. Quindi:

```
for (let giro = 0; giro < tentativi; giro++) {
  const seme = semeIniziale + giro * 7919;
  const piano = pianifica(voci, { ...opzioni, seme });
  // ...misura, tieni il più vicino, fermati se sei in tolleranza
}
```

Otto tentativi, nessuna logica nuova sui mazzi. E c'è un vantaggio non ovvio: ogni piano proposto è un piano che il generatore **avrebbe potuto produrre da solo**. Un ottimizzatore che lima carte per far quadrare un numero produce mazzi che nessuna regola giustifica.

Il seme si incrementa di 7919 (un primo) invece che di 1. Così due ricerche avviate da semi diversi esplorano zone diverse invece di sovrapporsi — che è ciò che fa continuare a funzionare il pulsante “Rigenera diversi”.

Il bug che solo l'app vera poteva mostrare

Chiedendo mazzi da 31 usciva un piano con un mazzo a `motore: 0` — le sue Energie non alimentavano nessuno dei suoi Pokémon. Non era un errore di calcolo: la ricerca stava facendo *esattamente* quel che le era stato chiesto. Un mazzo rotto ha una forza bassa, e puntando in basso è il candidato ideale.

È il classico effetto perverso di ottimizzare una metrica: **la metrica misura la forza, non la giocabilità**, e chi ottimizza sfrutta la differenza. La correzione non tocca la misura, aggiunge un vincolo:

```
function meglio(a, b) {
  if (a.giocabile !== b.giocabile) return a.giocabile; // prima la giocabilità
  return Math.abs(a.scarto) < Math.abs(b.scarto);      // poi la vicinanza
}
```

Un ordinamento **lessicografico**: la vicinanza al bersaglio decide solo fra piani entrambi giocabili. Effetto misurato sulla collezione di casa: da un mazzo a energie 0 a due mazzi a energie 58 e 46, al costo di un punto di distanza dal bersaglio.

Quando si ottimizza un punteggio, chiedersi sempre: *qual è il modo più stupido di massimizzarlo?*
Se esiste, prima o poi l'algoritmo lo troverà — e non lo troverà nei test, lo troverà in produzione.

Onestà del risultato

Con la collezione di casa il bersaglio 31 **non si raggiunge**: il meglio è 44. L'app lo dice, con la parola che serve:

Kit Allenatore Alola vale 31, i tuoi mazzi 44: un po' più forte. Avevi chiesto alla pari, ma con questa collezione non si è riusciti ad avvicinarsi di più.

Un'app che tacesse lascerebbe scoprire lo squilibrio perdendo una partita — cioè proprio il fallimento che tutta questa funzione doveva evitare.

Una nota su CSS e specificità

La tacca del riferimento sulla barra sembrava funzionare, e non funzionava:

```
.forza-barra > span { block-size: 100%; background: var(--colore-primario); }  
.tacca-riferimento { inset-block: -0.2rem; background: var(--colore-testo); }
```

.forza-barra > span ha specificità **(0,1,1)**, .tacca-riferimento **(0,1,0)**: la regola del riempimento vince, e la tacca ereditava altezza e colore sbagliati. Si vedeva qualcosa, quindi a occhio sembrava a posto — l'ho scoperto solo misurando `getBoundingClientRect()` (8,8 px invece di 15,2).

La soluzione non è `!important`, è togliere di mezzo il selettore discendente: il riempimento ha ora una classe sua, `.forza-riempimento`, e le due regole non si incontrano più.

Chi arriva da Angular ha l'incapsulamento di stile per default e questo problema non lo incontra quasi mai. In CSS globale la regola pratica è: **non selezionare per struttura ciò che puoi selezionare per nome**. Un `> span` cattura anche gli span che aggiungerai fra sei mesi.

10. Misurare mentre si costruisce

Il costruttore manuale (`src/app/vista-personalizzato.js`) è la stessa misura di prima usata al contrario: invece di generare un mazzo e dirti quanto vale, ti lascia scegliere e ti mostra il numero muoversi carta per carta. Tre decisioni di progetto che valgono più del codice.

Avvisi, non divieti

Il costruttore potrebbe impedire la quinta copia di una carta, o di scendere sotto il minimo di Base. Non lo fa, e non è pigrizia: in questo progetto **violare il regolamento in modo consapevole è la funzione principale** — le regole della casa nascono esattamente da lì. Un costruttore che impedisce di sbagliare impedirebbe anche di giocare come si gioca in casa.

Restano però due conversazioni diverse, e GRAVITA le tiene distinte:

gravità	significa	esempio
bloccante	il mazzo non è giocabile così	nessun Pokémon Base, nessuna Energia
avviso	si gioca, ma sappi che	2 Base su 30, evoluzioni orfane

Ogni avviso porta con sé **le carte a cui si riferisce**. «Ci sono evoluzioni orfane» non si può correggere; «Machamp (serve Machoke)» sì. È la differenza fra un messaggio e un'istruzione.

Il bug del 100%

Con due carte scelte, l'app annunciava: «Solo 1 Pokémon Base su 30 carte: con una mano da 7 hai il 100% di poter cominciare». Vero e assurdo insieme — pescando 7 carte da un mazzo di 2 le peschi tutte.

```
probabilitaAlmenoUna(totale, basi, mano) // sbagliato
probabilitaAlmenoUna(Math.max(totale, taglia), basi, mano) // giusto
```

La domanda non è «che probabilità ho **adesso**», ma «che probabilità avrò col mazzo finito». Mentre si costruisce, lo stato corrente non è mai la cosa da misurare — e questo vale ben oltre le carte Pokémon.

Il tetto che nascondeva due categorie su tre

L'elenco mostrava al massimo 60 righe, ordinate Pokémon □ Allenatori □ Energie. Con una collezione da 128 carte il risultato è che si vedevano **solo Pokémon**: Energie e Allenatori cadevano oltre il taglio e sparivano, senza nessun indizio che esistessero. Cioè le carte che si aggiungono più spesso.

La correzione è di una riga concettuale — il tetto diventa **per categoria** invece che complessivo — ma la lezione è sul modo di trovarla: non l'ha trovata un test, l'ha trovata provare a costruire un mazzo vero e non riuscire ad aggiungere un'Energia.

Regola pratica: un limite di visualizzazione applicato a una lista *ordinata per categoria* non tronca «gli ultimi elementi», tronca **le ultime categorie**. Se l'ordinamento raggruppa, il limite deve raggruppare con lui.

Shadow DOM e temi

<costruttore-mazzo> vive in Shadow DOM: i suoi selettori CSS non escono e quelli del documento non entrano. Ma le **custom property attraversano il confine**, perché sono ereditate:

```
.comandi button { background: var(--colore-superficie-2); }
```

Quel valore arriva da :root in base.css, quindi il componente segue il tema chiaro/scuro senza saperne niente. È il meccanismo previsto per far entrare un tema in un componente incapsulato — chi arriva da Angular conosce :ng-deep come scappatoia per lo stesso problema; qui non serve nessuna scappatoia.

11. Riparare i dati alla fonte, non nel punto in cui fanno male

Per tre sezioni questo documento ha detto «i set Kit Allenatore non hanno gli attacchi» come se fosse un fatto immutabile, e ha costruito difese intorno: *copertura*, *attendibile*, il messaggio «punteggio approssimato». Difese giuste — ma il buco resta, e si ripresenta in ogni funzione nuova. Il costruttore manuale mostrava *offesa 0* su un mazzo di Lycanroc e Raichu.

Il conto vero: **204 Pokémon su 12.877**, l'1,6%. Ma non sparsi a caso — concentrati nei Kit, cioè nelle carte con cui si gioca davvero. Un difetto dell'1,6% che colpisce il 100% dei casi d'uso.

E quelle carte **esistono complete altrove**: sono ristampe.

Dove metterlo

Tre posti possibili, e due sbagliati:

dove	cosa comporta
in forza()	il motore dovrebbe leggere il dataset, e <code>src/engine/</code> non tocca dati per principio
a runtime in <code>dataset.js</code>	cercare un omonimo vuol dire scorrere tutti i set: 6,4 MB scaricati per leggere un attacco, in un'app che carica un set per volta <i>apposta</i>
in uno strumento di sviluppo	la ricerca si fa una volta, il risultato è un file da 42 KB

`tools/completa-ristampe.mjs` produce `data/ristampe.json`, e `dataset.js` lo applica **al caricamento del set** — non alla singola lettura. Così ogni consumatore (griglia, motore, costruttore) vede le stesse carte senza doversi ricordare di chiamare qualcosa: la riparazione sta nel punto in cui i dati entrano nell'app, non in ognuno dei punti in cui servono.

Due errori commessi rifacendolo, entrambi nel merge

Primo: rompere una scelta già presa, rifattorizzando. La logica esisteva già dentro `genera-mazzi-prefatti.mjs`. Estraendola in `tools/lib/ristampe.mjs` per non averne due copie, ho cambiato senza accorgermene la struttura del ciclo: da «raccolgi tutte le omonime, poi preferisci quella con gli stessi PS» a «per ogni set, cerca». Sembra equivalente e non lo è — la preferenza per i PS diventa locale al set, e il Lycanroc del Kit (110 PS) è tornato a prendere gli attacchi del promo da 120 solo perché quel set viene prima nell'elenco.

Se ne sono accorti i **dati**, non un test: il file rigenerato aveva 59 righe diverse. Quando un refactor tocca un generatore, il diff dell'output è il test di regressione più economico che esista.

Secondo: `.length` non è «utilizzabile». Il merge teneva gli attacchi originali quando c'erano:

```
attacchi: carta.attacchi?.length ? carta.attacchi : dati.attacchi
```

Solo che le carte dei Kit hanno un array **pieno di voci senza nome e senza costo**. Presenti, quindi vincevano; e inservibili, perché `forza()` misura il danno *per Energia spesa*. A schermo comparivano attacchi `undefined` OE 10. Il criterio giusto non è «ne ha» ma «ne ha di misurabili»:

```
const suoiUsabili = (carta.attacchi ?? []).some((a) => (a.costo ?? []).length);
```

È lo stesso errore della sezione 4 — dato mancante $\neq$ zero — travestito da controllo di presenza. `array.length > 0` risponde a «esiste?», quasi mai a «serve?».

Dopo la correzione: 24 carte dei Kit su 24 con attacchi utilizzabili, offesa da 0 a 33, e il messaggio «punteggio approssimato» sparito.

12. Quando chi costruisce e chi misura non sono d'accordo

`proporzioni.js` riempiva i mazzi con **un terzo di Energie**. `forza.js` li giudicava volendone **costo medio × 0,11** — il 22% con attacchi da due Energie. Due regole diverse sulla stessa cosa, scritte in due momenti diversi, ciascuna sensata da sola.

L'effetto non è un errore visibile da nessuna parte: è una perdita costante.

costo medio	costruito	voluto	adeguatezza
1,5	33 %	17 %	0,49
2,0	33 %	22 %	0,74
2,5	33 %	28 %	0,89

Nel caso tipico ogni mazzo generato buttava via **un quarto del proprio motore**, che col peso 0,20 fa cinque punti di forza. Sempre, su tutti i mazzi, senza che nessun modulo sbagliasse i propri conti: si contraddicevano, e vinceva quello che misura.

La correzione non è “usare lo stesso numero”

La tentazione è copiare 0.11 anche in `proporzioni.js`. Sarebbe lo stesso bug con un anno di ritardo: due copie di una costante divergono alla prima taratura. La costante ora **vive in** `proporzioni.js` — il modulo che costruisce — e `forza.js` la importa. La direzione conta: chi misura si adegua a chi costruisce, non il contrario, perché la proporzione di un mazzo è una scelta di progetto e il punteggio è la sua verifica.

E la quota non è più fissa: `composizione()` riceve il costo medio degli attacchi delle carte con cui sta costruendo (`costoMedioAttacchi()`), quindi un mazzo di attacchi da una Energia ne riceve poche e uno di attacchi da tre ne riceve molte. Sulla collezione di casa il costo medio è **1,89** □ 21% di Energie invece del 33%.

Il test che lo blocca non confronta le costanti

Sarebbe una tautologia: sono la stessa variabile importata, l'uguaglianza è garantita dal linguaggio. Il test confronta il **risultato**:

```
const quota = composizione(taglia, abbondanza, { costoMedio: costo });
const m = mazzoConQuellaComposizione(quota);
assert.ok(forza(m, { taglia }).motore > 0.85);
```

Cioè: *un mazzo costruito secondo le nostre proporzioni dev'essere giudicato bene dal nostro misuratore*. Vale su 4 taglie × 3 costi, e continuerebbe a valere anche se un domani le due formule divergessero di nuovo per altra strada.

Scrivendolo ha subito trovato un residuo: il pavimento della quota Energie era al 15% mentre con attacchi da una sola Energia ne bastava l'11% — lo stesso disaccordo, in piccolo, appena reintrodotta da un numero messo lì per prudenza. Ora il pavimento **è** la quota di un attacco da una Energia.

Due correzioni minori, trovate rileggendo

`piramide()` aveva un ramo morto: `taglia >= 40` e `taglia >= 25` restituivano entrambi `[3, 2, 1]`. Il primo `if` non faceva niente da sempre. Il 60 ora vuole `[4, 3, 2]`: in un mazzo doppio la stessa linea si pesca la metà delle volte, quindi non raddoppiarla significa metterci una linea che quasi non entra in gioco.

`minimoBasi()` chiedeva il 25% fisso, cioè **15 Pokémon Base su 60** — i mazzi veri ne hanno 8-12. Quei tre slot sono esattamente la differenza fra un mazzo con due linee evolutive complete e uno con una sola. Ora è un quinto sopra le 30 carte, e la probabilità di aprire regge lo stesso (12 Base su 60, mano da 7: 80%).

Esercizi

1. **La scala satura.** Nei mazzi generati da una collezione moderna, *offesa* arriva spesso a 1.00: il tetto di 55 viene raggiunto. Misura sul dataset il p95 e il p99 del danno per Energia (`node -e`, come nella sezione 3) e ragiona: alzare il tetto renderebbe la scala più informativa, o schiaccerebbe verso il basso i mazzi di casa che sono il caso d'uso vero? Prova a cambiare `TETTI.dannoPerEnergia` e rilancia i test: **quali si rompono, e quali no?** Quelli che non si rompono sono i test di proprietà della sezione 2.
2. **Un indicatore in più.** Il costo di ritirata (`carta.ritirata`, presente sul 99,9% dei Pokémon) misura quanto un mazzo resta bloccato con la carta sbagliata in campo. Aggiungi `mobilita` a `forza()`: decidi il tetto *misurandolo*, non scegliendolo, e riequilibra `PESI` in modo che la somma resti 1. Scrivi prima il test che dice cosa ti aspetti.
3. **La domanda scomoda.** `struttura` si ferma a 0,20-0,25 su quasi tutti i mazzi generati (guarda i valori nella sezione 5 di questo documento). Un indicatore che non varia mai non distingue niente e sta solo diluendo gli altri. È un difetto della misura — il divisore 2 è troppo generoso — o un fatto vero sui mazzi che il motore produce? Come faresti a **distinguere le due ipotesi** senza cambiare il codice?